# Pedal Controller: Architecture and Task Split

## 1 Overview

The board is the control part of a guitar pedal. It reads three knobs and two buttons, and it sets three digital potentiometers inside an external audio unit. Audio never passes through the board.

The firmware runs on the STM32F407G-DISC1. In live mode the knobs set the digital pots directly. The user can also recall one of two saved presets, edit it with the knobs and save it again. The four LEDs on the board show the state.

The structure, the task split and the interfaces below are a first draft.

## 2 Components

The build needs eight kinds of parts (Table 1). The digital pots and the footswitch are still missing.

| Part | Qty | Job | Connects to | Status |
|---|---|---|---|---|
| STM32F407G-DISC1 | 1 | Runs the firmware: ADC, SPI, GPIO, flash for presets | USB to the PC for power, flashing and debug | One at home, lab boards for the others |
| Physical potentiometer | 3 | Knobs P1, P2, P3 | ADC input pins | One per student so far, three needed |
| Digital potentiometer module | 3 | DP1, DP2, DP3: set the audio unit's three parameters | SPI bus (proposed) | Part not chosen yet |
| Footswitch B1 | 1 | Short press: standby on/off. Long press: change preset | GPIO input | To get |
| Button B2 | 1 | Long press: save the preset | GPIO input | The board's user button (PA0) can stand in |
| LEDs on the board | 4 | Green: board on, dimmed in standby. Orange: Preset 1. Red: Preset 2. Blue: preset edited, not saved | Built in: PD12 green, PD13 orange, PD14 red, PD15 blue | On the board |
| Breadboard and jumper wires | 1 set | Prototype wiring | All external parts | To check |
| Logic analyzer (test tool) | 1 | Checks the SPI bytes and the LED timing | Probes on the SPI and LED pins | One at home |

Table 1: Hardware parts.

The board's own LEDs show the state, so no external LEDs are needed. The board's user button allows the firmware to be tested before any external part is wired. The button functions follow the README and the flow chart; see Section **??**.

## 3 Architecture

The firmware has three layers, and calls only go down (Figure 1).

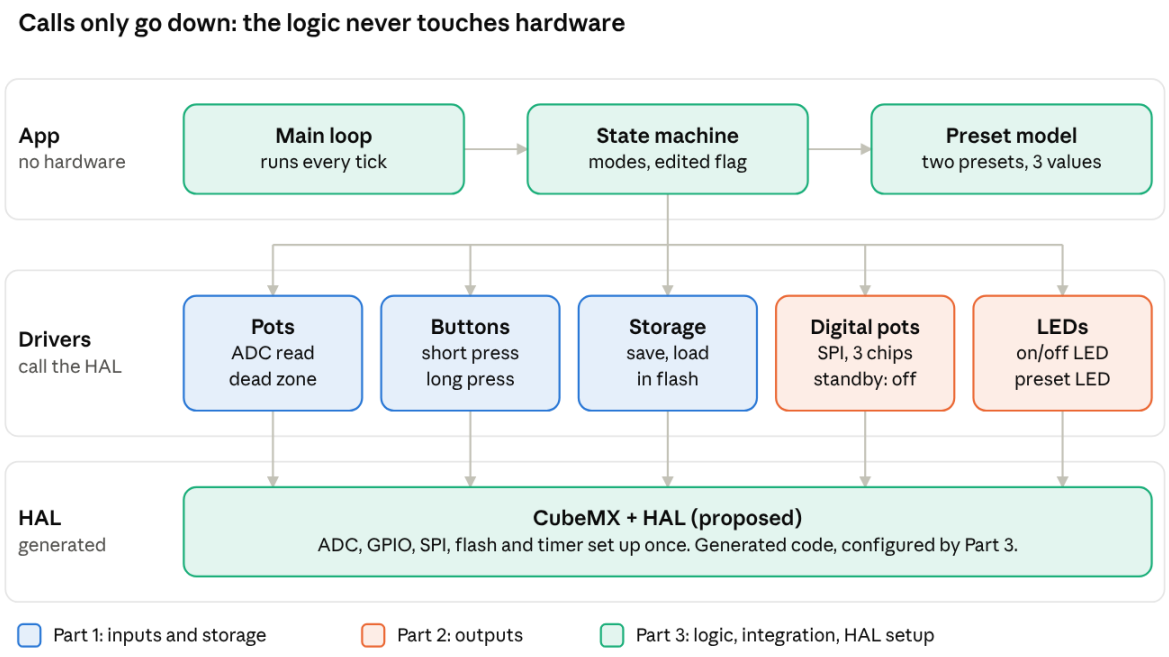



Figure 1: Firmware layers. Box color shows which part owns each module.

- **App:** the main loop, the state machine and the preset data. It has no hardware access, so it builds and runs on a PC for testing.
- **Drivers:** one small module per piece of hardware. These are the only files that call the HAL.
- **HAL:** the code CubeMX generates from the project's `.ioc` file.

The rule pays off twice. The state machine can be tested on any PC before the hardware is wired. A part can change, for example the digital pot chip, without touching the logic.

## 4 How it works

The board is always in one of four states: Live, Preset 1, Preset 2 or Standby (Figure 2).

**Power-up.** The board reads the three knobs and writes those values to the digital pots. It starts in Live with only the green LED on.

**Live.** Every knob move goes straight to its digital pot.

**Changing mode.** A B1 long press moves Live to Preset 1, Preset 1 to Preset 2, and Preset 2 back to Live. Loading a preset writes its three stored values to the digital pots. The orange LED lights for Preset 1 and the red LED for Preset 2. Both are off in Live.

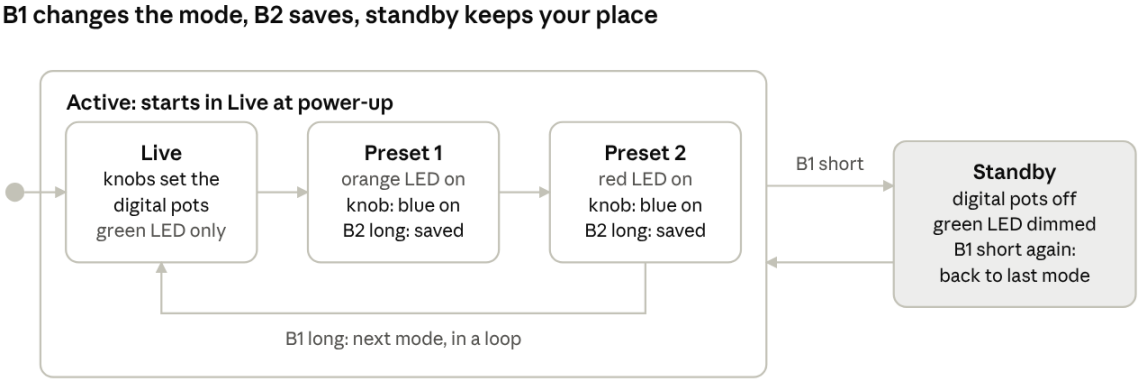



Figure 2: State machine. Button events are on the arrows.

**Editing a preset.** Turning a knob in a preset moves that digital pot to the knob's position and marks the preset as edited. The blue LED stays on until the preset is saved.

**Saving.** A B2 long press in a preset writes its three values to flash. The edited mark clears, and the blue LED blinks fast three times and goes off. B2 does nothing in Live.

**Standby.** A B1 short press turns the digital pot outputs off. The green LED dims and the other LEDs go off. The power stays on, and the next B1 short press returns to the mode the board left, so a selected preset stays selected.

**Button timing.** A press shorter than about 1 s is a short press. Between 1 and 3 s nothing happens, so a wrong function cannot fire by accident. A press of 3 s or more is a long press.

**Main loop.** The board runs on a fixed tick from a timer. On every tick the main loop calls the state machine once. The state machine collects new events from the input drivers and then updates the output drivers. The tick length is set in the detailed design.

**Presets in flash.** Two presets, three values each, live in one flash sector reserved for them. The sector is picked once the size of the build is known.

## 5 Task split

The work splits into three parts of about the same size (Table 2). Each owner builds their part end to end: wiring, code, a test, and their section of the report.

Part 2 needs a board at home. Parts 1 and 3 work in the lab, and Part 3's logic runs on any PC.

### Part 1: Inputs and storage

- ☐ Wire one pot to an ADC pin and read its value
- ☐ Read all three pots, with a dead zone so noise does not count as a knob move
- ☐ Button driver for B1 and B2: short press, the 1 to 3 s gap, long press
- ☐ Storage: save and load two presets of three values in the reserved flash sector
- ☐ Test: a saved preset is still there after a power cycle

| Part | Builds | Tested where | Shows at the demo | Owner |
|---|---|---|---|---|
| 1. Inputs and storage | Knob reading, button timing, preset save and load in flash | Lab board. One pot and the board's user button are enough to start | Knob values and button events in the debugger. A preset that survives a power-off | To agree |
| 2. Outputs | Digital pot driver over SPI, LED patterns, outputs off in standby | At home on a DISC1. The SPI bytes show on the logic analyzer before the chips arrive | A meter on each wiper. The SPI bytes on the analyzer | To agree |
| 3. Logic and integration | CubeMX project, main loop, state machine, preset model | Any PC for the logic. Lab board for the full build | The whole pedal working | To agree |

Table 2: The three parts.

### Part 2: Outputs

- ☐ SPI driver: set one digital pot. Check the bytes on the logic analyzer against the datasheet
- ☐ Drive all three chips, and turn the outputs off in standby
- ☐ LED driver for the four board LEDs: green on, or dimmed in standby with timer PWM; orange or red for the preset, blue while edited, three fast blue blinks on save
- ☐ Test: a meter on each wiper while stepping the value

### Part 3: Logic and integration

- ☐ CubeMX project (`.ioc`) with every pin from the pin list, replacing the empty template
- ☐ Main loop on a fixed timer tick
- ☐ State machine: the four states, the edited flag, power-up
- ☐ Preset model: two presets in RAM, handed to Storage on save
- ☐ Test the state machine on a PC with made-up button events
- ☐ Join Parts 1 and 2 and run the full test

## 6 How the parts connect

The three parts meet only through the functions in Table 3. If each part keeps its side of this table, the parts can be built apart and joined at the end.

`ch` is 1, 2 or 3, matching P1 to P3 and DP1 to DP3. Every knob and pot value uses the ADC's own range, and Part 2 converts it to the chip's steps. A new chip then changes one file. The shared names, the value range and the tick length live in one header, `pedal.h`. Part 3 keeps it, and any change is agreed by all three.

Figure 3 shows one example: knob 2 is turned in Preset 1, and then the preset is saved. One tick at a time, the state machine asks the inputs and then sets the outputs. Dashed arrows are answers.

| Function | Owner | Called by | What it does |
|---|---|---|---|
| `pots_read(v)` | Part 1 | State machine | Fills `v` with the three knob values |
| `pot_moved(ch)` | Part 1 | State machine | True if knob `ch` moved past the dead zone since the last check |
| `button_event()` | Part 1 | State machine | Returns the next event: none, B1 short, B1 long or B2 long |
| `storage_save(p, v)`, `storage_load(p, v)` | Part 1 | State machine | Writes or reads preset `p` (1 or 2), three values |
| `digipot_set(ch, value)` | Part 2 | State machine | Sets digital pot `ch` to `value`, scaled to the chip's steps |
| `digipot_enable(on)` | Part 2 | State machine | Turns all three outputs on or off |
| `leds_set(power, preset, edited)` | Part 2 | State machine | Green on, or dimmed in standby. Orange or red for the preset, blue while edited |
| `leds_saved()` | Part 2 | State machine | Starts the three fast blue blinks |
| `leds_tick()` | Part 2 | State machine | Runs the blink timing, once per tick |
| `sm_init()`, `sm_tick()` | Part 3 | Main loop | Starts the state machine, then runs it once per tick |

Table 3: Interfaces between the parts.

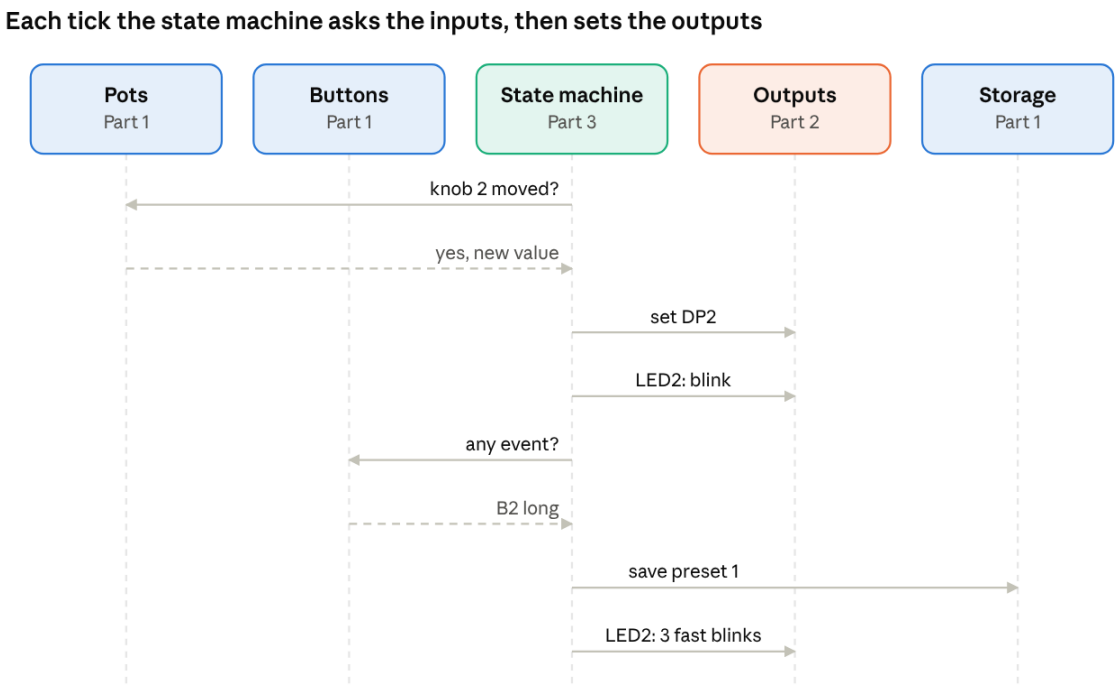



Figure 3: Call sequence: knob 2 turned in Preset 1, then the preset saved.

### 6.1 Integration order

1. Discuss further and agree on this document, the pin list and `pedal.h`.
2. Each part works alone. Parts 1 and 2 on a board, Part 3 on a PC with fake drivers.
3. Part 3 joins Part 1: the knobs and buttons drive the state machine.
4. Part 2 joins: the full build runs on one board.
5. Run the full test, take the documentation photos.

### 6.2 Pins and the `.ioc` file

Part 3 owns the `.ioc` file. Parts 1 and 2 send the pins they need, and Part 3 enters them in CubeMX. SPI1 already talks to the board's own accelerometer, so the digital pots get SPI2 or SPI3 (to be discussed).

### 6.3 Files and git

- CubeMX folders as generated, plus `App/` for Part 3 and `Drv/` for the drivers.
- Part 1: `pots.c`, `buttons.c`, `storage.c`. Part 2: `digipot.c`, `leds.c`.